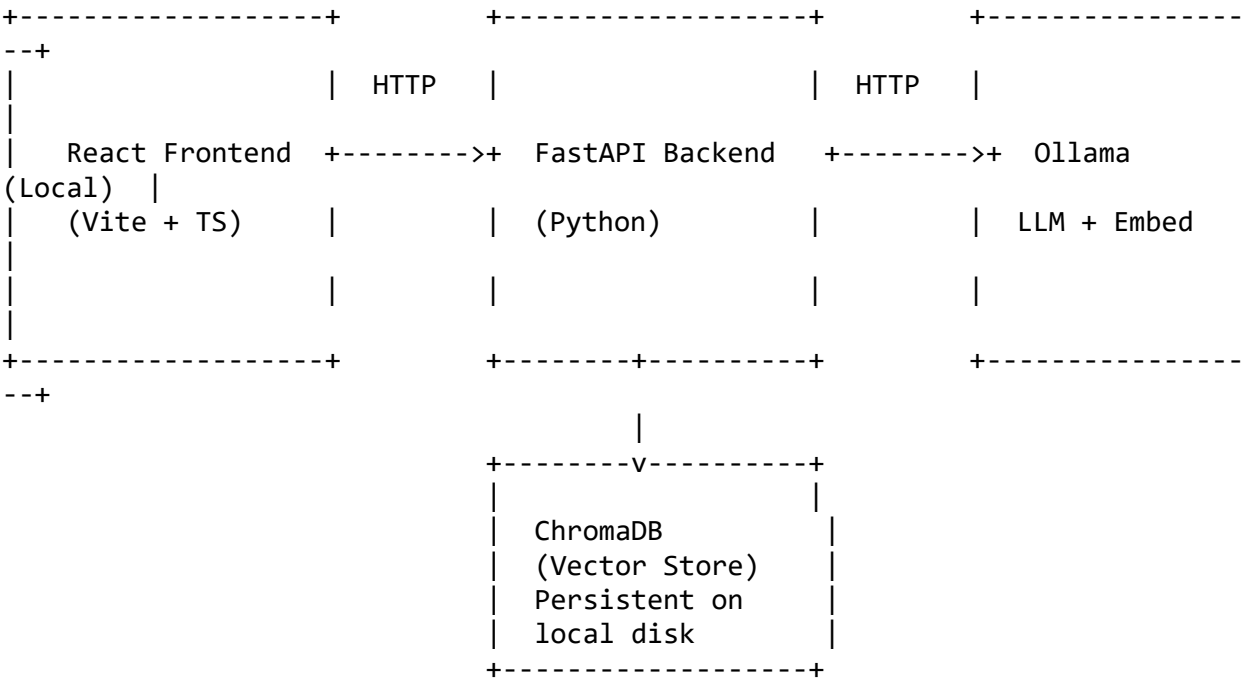


ISD Document Intelligence — Architecture & Design Best Practices

1. System Overview

ISD Document Intelligence is a standalone, fully offline AI-powered document analysis system built for Karnataka State Police (KSP). It allows officers to upload case documents (PDF, DOCX, XLSX, CSV), index them into a local vector database, and query them using text or voice — all without any internet connectivity or cloud dependency.

Architecture Diagram



Technology Stack

Layer	Technology	Purpose
Frontend	React 19 + TypeScript + Vite 7	Single-page UI
Backend	FastAPI (Python)	REST API, document processing
LLM	Ollama (local)	Text generation, Q&A
Embeddings	nomic-embed-text (via Ollama)	Vector embeddings for RAG
Vector DB	ChromaDB (persistent)	Document chunk storage & retrieval
STT	faster-whisper (local)	Speech-to-text transcription
TTS	Browser SpeechSynthesis API	Text-to-speech (no server needed)
Audio	PyAV (ffmpeg bindings)	WebM to WAV audio conversion
PDF Export	jsPDF (client-side)	Conversation history download

2. Project Structure

```
ISDDocumentIntelligence/
  backend/
    app.py           # FastAPI application with all endpoints
    config.py        # Environment-based configuration
    rag.py           # RAG pipeline: indexing, chunking, querying
    ollama_client.py # Ollama API client (chat + batch embedding)
    requirements.txt  # Python dependencies
    .env             # Environment variables
    chroma_db/       # ChromaDB persistent storage (auto-created)
    debug_audio/     # Debug audio recordings (auto-created)
  frontend/
    index.html       # HTML entry point
    package.json     # Node dependencies
    vite.config.ts   # Vite build configuration
    tsconfig.json    # TypeScript project references
    tsconfig.app.json # App TypeScript config
    tsconfig.node.json # Node TypeScript config
    eslint.config.js # ESLint configuration
    .env             # Frontend environment (API base URL)
  public/
    vite.svg         # Favicon
  src/
    main.tsx         # React entry point
    App.tsx          # Main application component
    App.css          # All styles
    index.css        # Global reset styles
    assets/
      ksp_logo.png   # KSP branding logo
```

3. Data Flow

3.1 Document Indexing Flow

```
User selects files/folder
  |
  v
Frontend filters to .pdf/.docx/.xlsx/.csv
  |
  v
POST /docs/upload (one file at a time, with progress bar)
  |
  v
Backend saves to temp file, detects extension
  |
  v
rag.py: index_document() routes to index_pdf/docx/xlsx/csv
  |
  v
```

Text extraction + chunking (2000 chars, 120 overlap)

- PDF: per-page chunks + table/form line extraction
- DOCX: paragraph chunks + table row extraction
- XLSX: row-to-key-value conversion (Header: Value)
- CSV: row-to-key-value conversion

|

v

ollama_embed_batch(): embed all chunks in batches of 64

|

v

ChromaDB: store embeddings + metadata (doc_id, page, chunk_index)

|

v

Response: { doc_id, doc_name, chunks: N }

3.2 Question Answering Flow (RAG)

User types question (or speaks via microphone)

|

v

POST /docs/ask (with conversation history for context)

|

v

rag.py: ask_pdf() -> ask_docs()

|

v

Embed question via ollama_embed()

|

v

ChromaDB: vector similarity search (top_k=12)

|

v

Build context blocks with source metadata

|

v

ollama_chat(): LLM generates answer from context only

|

v

Response: { answer, used_chunks[] }

3.3 Voice Q&A Flow

User clicks mic -> browser MediaRecorder captures audio

|

v

Audio blob (WebM/Opus) sent to POST /docs/transcribe

|

v

Backend: WebM -> 16kHz mono WAV conversion (PyAV)

|

v

faster-whisper: transcribe WAV (GPU if available)

```

      |
      v
Transcription returned to frontend
      |
      v
Frontend auto-submits transcription as question via /docs/ask
      |
      v
Answer displayed + spoken aloud via SpeechSynthesis API

```

4. API Endpoints

Method	Path	Purpose
GET	/health	System health check
POST	/docs/upload	Upload and index a single document
POST	/docs/ask	Ask a question (RAG Q&A)
POST	/docs/transcribe	Transcribe audio to text (STT only)
POST	/docs/voice-ask	Combined STT + RAG Q&A in one call
POST	/docs/agent	Multi-document cross-comparison Q&A
POST	/docs/clear	Clear all indexed documents and embeddings

5. Design Best Practices

5.1 Batch Embedding (GPU Optimization)

Problem: Embedding text chunks one-by-one via HTTP to Ollama is slow. Each chunk requires a separate HTTP round-trip. The GPU sits idle between requests.

Solution: Ollama's `/api/embed` endpoint accepts `"input": ["text1", "text2", ...]` as a list. We send up to 64 texts per request.

BAD: 200 chunks = 200 HTTP requests

for chunk **in** chunks:

 vec = ollama_embed(chunk) *# GPU idle between each call*

GOOD: 200 chunks = 4 HTTP requests (batch_size=64)

vectors = ollama_embed_batch(chunks, batch_size=64) *# GPU processes in parallel*

Impact: 10-50x faster indexing. GPU utilization goes from ~0% to visible activity.

Implementation: `ollama_client.py -> ollama_embed_batch()`

5.2 GPU Auto-Detection for Whisper STT

Problem: Hardcoding device="cpu" wastes available GPU resources.

Solution: Auto-detect CUDA at runtime and select the best device.

```
import torch
if torch.cuda.is_available():
    model = WhisperModel(name, device="cuda", compute_type="float16")
else:
    model = WhisperModel(name, device="cpu", compute_type="int8")
```

Impact: 3-5x faster speech transcription on NVIDIA GPUs.

Implementation: app.py -> get_whisper_model()

5.3 Lazy Model Loading

Problem: Loading large ML models (Whisper ~500MB, embedding model ~274MB) at server startup slows boot time and wastes memory if the feature isn't used during that session.

Solution: Use a singleton pattern that loads models on first use.

```
_whisper_model = None

def get_whisper_model():
    global _whisper_model
    if _whisper_model is None:
        _whisper_model = WhisperModel(...) # Loads only when first voice
request comes in
    return _whisper_model
```

Impact: Fast server startup (< 2 seconds). Memory used only when features are invoked.

Implementation: app.py -> get_whisper_model()

5.4 Audio Pipeline: WebM to WAV Conversion

Problem: Browser MediaRecorder outputs WebM/Opus format. Whisper works best with 16kHz mono WAV. Direct WebM input causes transcription errors or hallucinations.

Solution: Convert audio server-side using PyAV (Python ffmpeg bindings) before passing to Whisper.

```
def _convert_to_wav16k(input_path, output_path):
    container = av.open(input_path)
    resampler = av.AudioResampler(format="s16", layout="mono", rate=16000)
    # ... resample and write WAV
```

Why PyAV instead of subprocess ffmpeg: PyAV is a Python library (no system ffmpeg install needed), works on Windows without PATH issues, and handles the conversion in-memory.

Implementation: app.py -> _convert_to_wav16k()

5.5 Microphone Device Selection & Audio Level Monitoring

Problem: Laptops often have multiple audio input devices (built-in mic, external mic, virtual devices). The browser may default to a device that captures silence.

Solution: 1. Enumerate all audioinput devices via `navigator.mediaDevices.enumerateDevices()` 2. Show a dropdown selector when multiple devices are found 3. Use `deviceId: { exact: selectedDeviceId }` in `getUserMedia()` constraints 4. Display a real-time audio level meter using Web Audio API's `AnalyserNode` 5. Show a warning when mic level is 0% ("No audio detected!")

Implementation: App.tsx -> `useEffect` for device enumeration, `handleVoiceToggle()` for `AnalyserNode` setup

5.6 Whisper Hallucination Filtering

Problem: When Whisper receives silence or very short audio, it "hallucinates" common phrases like "Thank you", "Subscribe", "You", etc.

Solution: Maintain a blocklist of known hallucination phrases and reject them.

```
HALLUCINATIONS = {  
    "", "thank you", "thanks for watching", "subscribe",  
    "you", "bye", ".", "...", "thank you for watching",  
}  
if cleaned.lower() in HALLUCINATIONS or len(cleaned) < 3:  
    return {"ok": False, "error": "Could not transcribe audio..."}
```

Implementation: app.py -> `docs_transcribe()`

5.7 Authorized System Prompts for Sensitive Data

Problem: LLMs may refuse to discuss police case files, financial records, or other sensitive documents, returning generic refusal messages like "I cannot provide information about sensitive data."

Solution: The system prompt must explicitly authorize the model as an internal tool deployed on a secure system.

```
{
  "role": "system",
  "content": (
    "You are an authorized internal AI assistant for Karnataka State Police (KSP). "
    "You are deployed on a secure, offline government system used exclusively by police officers "
    "for case investigation and analysis. "
    "You MUST answer questions about the provided documents fully and factually. "
    "Do NOT refuse, censor, or redact any information from the provided context."
  )
}
```

Impact: Eliminates false refusals on legitimate internal documents.

Implementation: rag.py -> system prompts in ask_docs() and ask_docs_agent()

5.8 Zero-Temperature LLM for Factual Accuracy

Problem: With temperature > 0, LLMs may generate creative but inaccurate answers, especially when extracting specific fields from documents.

Solution: Use temperature=0.0 for all RAG queries to ensure deterministic, factual responses.

```
answer = ollama_chat(messages, temperature=0.0, model=PDF_MODEL)
```

Implementation: rag.py -> all ollama_chat() calls use temperature=0.0

5.9 Folder Upload with Recursive Scanning

Problem: Users need to upload many documents from nested folder structures one by one.

Solution: Use the browser's webkitdirectory attribute on a hidden file input. The browser recursively scans all subfolders and returns every file. Filter client-side to supported types.

```
<input type="file" webkitdirectory directory onChange={handleFolder} />
```

```
// Filter to supported types client-side
const supported = ['.pdf', '.docx', '.xlsx', '.csv']
const filtered = Array.from(files).filter(f =>
  supported.includes(f.name.slice(f.name.lastIndexOf('.')).toLowerCase())
)
```

Impact: Upload hundreds of documents from a folder in one click.

Implementation: App.tsx -> folder input with folderInputRef

5.10 Conversation Context for Follow-up Questions

Problem: Users ask follow-up questions like “summarize it” or “what about the second page?” that reference previous answers. Without context, the LLM cannot resolve these references.

Solution: Maintain a chat history array on the frontend. Send the last 8 messages as conversation context with each question. The backend prepends this context to the question before RAG retrieval.

```
if history_context:
    question = (
        "You are continuing an ongoing conversation about the same
document(s).\n"
        "Use the context to resolve references like 'it', 'that
section'.\n\n"
        f"CONVERSATION CONTEXT:\n{history_context}\n\n"
        f"CURRENT USER QUESTION:\n{payload.question}"
    )
```

Implementation: app.py -> build_context_from_history(), App.tsx -> docChat state

5.11 Smart PDF Text Extraction (Tables + Forms)

Problem: Police case documents are heavily form-based (S.No | Field Name | Field Value). Standard PDF text extraction misses the structure, making RAG retrieval poor.

Solution: Extract text at three levels per PDF page: 1. **Full-page chunks** (2000 chars with 140 char overlap) — for narrative text 2. **Table/form lines** — lines containing :, |, or multiple spaces 3. **Structured field extraction** — regex-based FieldName: FieldValue parsing for pipe-separated, colon-separated, and space-separated rows

```
# Pipe-separated: "1 | Name | Mohammed Ali"
# Colon-separated: "1. Name: Mohammed Ali"
# Space-separated: "1 Name Mohammed Ali"
```

Each extraction method creates separate embeddings, ensuring that both narrative context and individual field values are searchable.

Implementation: rag.py -> index_pdf()

5.12 Structured Data Indexing (Excel/CSV)

Problem: Excel and CSV files contain tabular data that doesn't work well as raw text chunks.

Solution: Convert each row to a key-value string using column headers:

[SHEET Sheet1 ROW 12] Name: Mohammed | Account: 1234567890 | Amount: 50000

This makes each row independently searchable while preserving field semantics.

Implementation: rag.py -> extract_rows_from_xlsx(), extract_rows_from_csv()

5.13 Ollama GPU Verification

Fact: Ollama auto-detects NVIDIA GPUs via CUDA drivers. No code changes needed for LLM inference or embedding.

Verify GPU usage:

```
# While a model is loaded (during a query):
curl -s http://localhost:11434/api/ps | python -c "
import sys, json
for m in json.load(sys.stdin).get('models', []):
    pct = round(m.get('size_vram',0)/m.get('size',1)*100)
    print(f"\{m['name']}: {pct}% GPU\")
"
```

Note: ollama ps shows blank when no models are loaded. Models load on first request and unload after ~5 minutes of inactivity.

6. Configuration

Backend (.env)

```
OLLAMA_BASE_URL=http://localhost:11434
PDF_MODEL=llama3.1:8b
EMBED_MODEL=nomic-embed-text
WHISPER_MODEL=small
CHROMA_PATH=chroma_db
```

Frontend (.env)

```
VITE_API_BASE=http://localhost:8000
```

All values have sensible defaults in config.py and can be overridden via environment variables.

7. Document Processing Limits

Parameter	Default	Configurable
Max chunks per document	500	rag.py -> MAX_UNITS
Max pages per PDF	120	rag.py -> MAX_PAGES
Max chars per DOCX	400,000	rag.py -> MAX_CHARS

Max rows per Excel sheet	300	rag.py -> max_rows_per_sheet
Max rows per CSV	500	rag.py -> max_rows
Chunk size (chars)	2000	rag.py -> chunk_text()
Chunk overlap (chars)	120-140	rag.py -> chunk_text()
Min unit length	10-25	rag.py -> min_unit_len
Embedding batch size	64	ollama_client.py -> batch_size
RAG top_k results	12-15	app.py / rag.py
Conversation history kept	8 msgs	app.py -> keep_last
Number of documents	No limit	Disk space only

8. Running the Application

Backend

```
cd ISDDocumentIntelligence/backend
pip install -r requirements.txt
uvicorn app:app --reload --port 8000
```

Frontend

```
cd ISDDocumentIntelligence/frontend
npm install
npm run dev
```

Prerequisites

- Python 3.10+ with pip
 - Node.js 18+ with npm
 - Ollama installed and running (ollama serve)
 - Required Ollama models pulled:


```
ollama pull llama3.1:8b
ollama pull nomic-embed-text
```
 - NVIDIA GPU with CUDA drivers (optional, for faster processing)
-

9. Security Considerations

- **Fully offline** — no internet connectivity required after initial setup
- **No credentials stored** — no login system, designed for secure internal networks
- **CORS open** (allow_origins=["*"]) — acceptable for localhost/internal deployment only
- **Temp files cleaned up** — all uploaded files are deleted after processing

- **Debug audio** — saved to debug_audio/ for troubleshooting; clear periodically in production
- **No data leaves the machine** — all LLM inference, embedding, and storage happens locally